



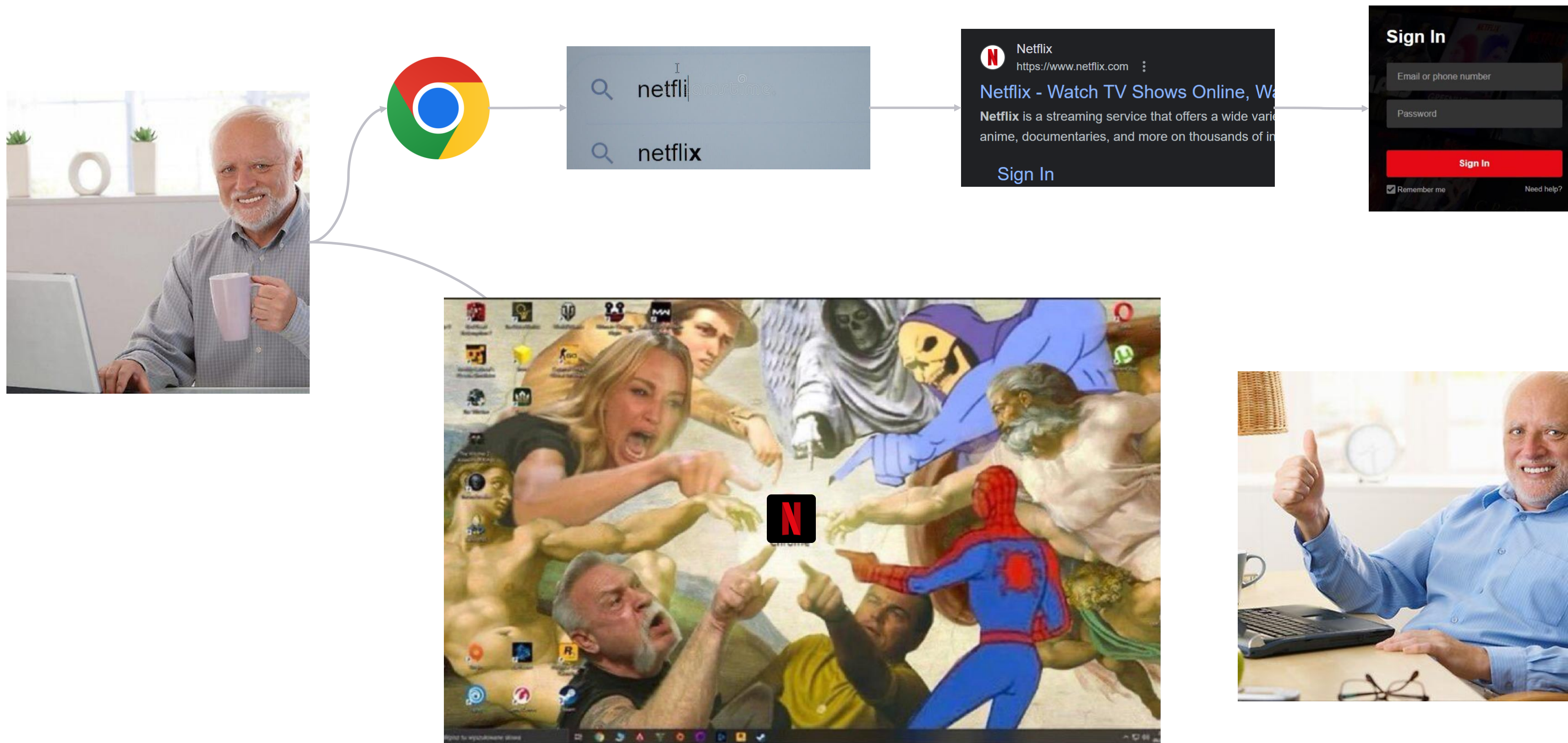
# Frameworks de Deep Learning

Redes Neuronales y Aprendizaje Profundo

01

# Niveles de Abstracción

# Mi abuelo y Netflix



# Niveles de abstracción



CAPAS ALTAS  
Mayor nivel de abstracción  
Más simplicidad



CAPAS BAJAS  
Menor nivel de abstracción  
Más flexibilidad

# Niveles de abstracción

Python



```
● ● ●  
1  num1 = 5  
2  num2 = 7  
3  result = num1 + num2  
4  print(f"Resultado: {result}")  
5
```

# Niveles de abstracción

C



```
1  #include <stdio.h>
2
3  int main() {
4      int num1 = 5;
5      int num2 = 7;
6      int result = num1 + num2;
7      printf("Resultado: %d\n", result);
8      return 0;
9  }
10
```

# Niveles de abstracción

## Ensamblador



```
1  section .data
2      num1    dd 5
3      num2    dd 7
4      result  dd 0
5
6  section .text
7      global _start
8
9  _start:
10     mov eax, [num1]    ; mover num1 a eax
11     add eax, [num2]    ; sumar num2 a eax
12     mov [result], eax ; guardar resultado
13
14     ; código para terminar el programa correctamente
15     mov eax, 1          ; syscall número 1: exit
16     xor ebx, ebx        ; código de salida 0
17     int 0x80
18
```

# ¿Una red neuronal en Python?



- Capas de Neuronas
- Cálculo de Sumas Ponderadas
- Funciones de Activación
- Propagación de la información hacia delante
- Cálculo del Error
- Propagación de Errores hacia atrás
- Cálculo de Derivadas
- Optimización de Parámetros con Descenso del Gradiente

Abstracción

- Difícil Experimentación
- Complejidad
- Coste de tiempos
- Mantenimiento



# Niveles de abstracción

## Tensorflow

```
1 import tensorflow as tf
2 from tensorflow.keras import Sequential
3 from tensorflow.keras.layers import Dense
4
5 model = Sequential([
6     Dense(4, activation='relu', input_shape=(3,)), # Capa oculta 1
7     Dense(4, activation='relu'),                  # Capa oculta 2
8     Dense(2, activation='softmax')                 # Capa de salida
9 ])
10
11 model.summary()
```

```
1 import random
2 import math
3
4 # Funciones de activación
5 def relu(x):
6     return [max(0, val) for val in x]
7
8 def dot_product(weights, inputs):
9     return [sum(w * i for w, i in zip(neuron, inputs)) for neuron in weights]
10
11 # Inicialización aleatoria
12 def init_weights(n_neurons, n_inputs):
13     return [[random.uniform(-1, 1) for _ in range(n_inputs)] for _ in range(n_neurons)]
14
15 # Red con pesos aleatorios
16 inputs = [0.5, 0.3, 0.9]
17
18 # Capa 1
19 weights1 = init_weights(4, 3)
20 layer1 = relu(dot_product(weights1, inputs))
21
22 # Capa 2
23 weights2 = init_weights(4, 4)
24 layer2 = relu(dot_product(weights2, layer1))
25
26 # Capa de salida
27 weights_out = init_weights(2, 4)
28 output = dot_product(weights_out, layer2)
29
30 print("Salida:", output)
31
32
```

02

# Frameworks

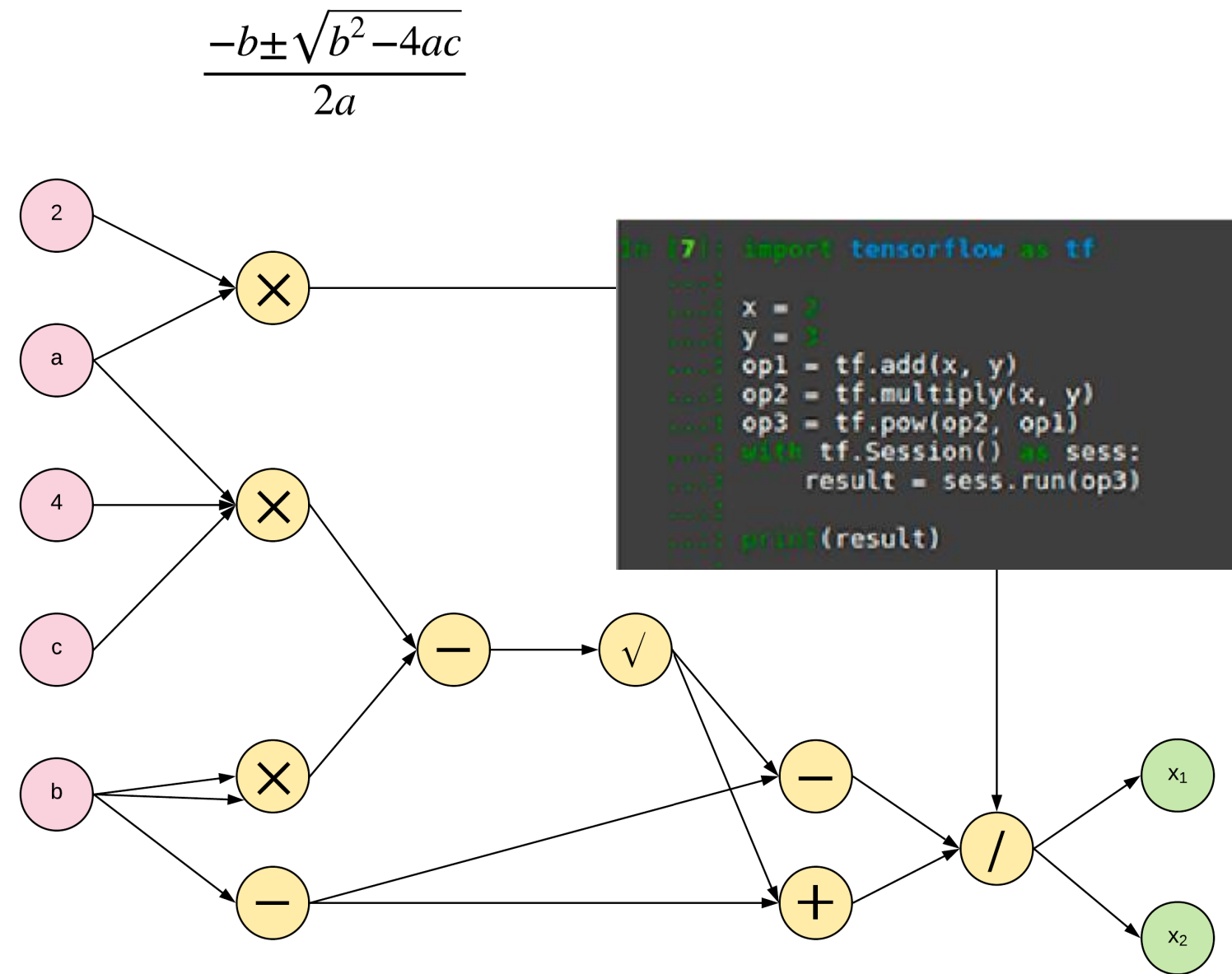
# Frameworks

- Conjunto de librerías de programación que facilita el desarrollo e implementación eficiente de redes neuronales (en el caso de DL).
- Gracias a la popularización del deep learning, ha sido el surgimiento de una gran cantidad de paquetes software para el entrenamiento de redes neuronales.

# Grafos de Computación

- La mayoría de frameworks entienden las redes neuronales como un grafo de computación, donde cada nodo es una operación.
- Los frameworks nos dan una serie de abstracciones y operaciones ya disponibles para que podamos construir dicho grafo de computación, para facilitar el desarrollo.

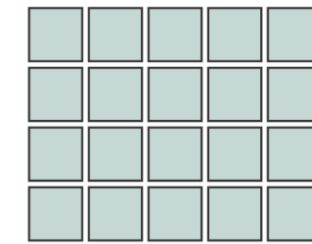
## Grafos de computación



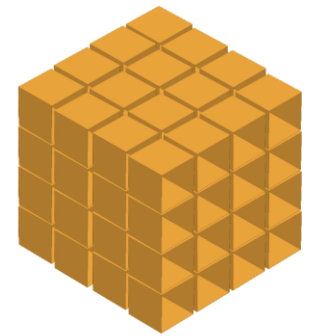
## Tensoros



RANK-1 TENSOR



RANK-2 TENSOR



RANK-3 TENSOR



#### Puntos Clave:

- Desarrollado por Google Brain.
- Soporta múltiples plataformas.
- Amplia gama de herramientas para desarrollar modelos de aprendizaje profundo.
- Escalable y apto para empresas.
- Ofrece TensorFlow Lite para dispositivos móviles y embebidos.
- Soporta procesamiento en la nube, en el borde y en el dispositivo.
- Extensa comunidad y documentación.
- Compatible con Keras para una programación de alto nivel.

#### Ventajas:

- Escalable y Versátil.
- Robusto y Apto para Investigación y Producción.
- Amplia Comunidad de Desarrolladores.



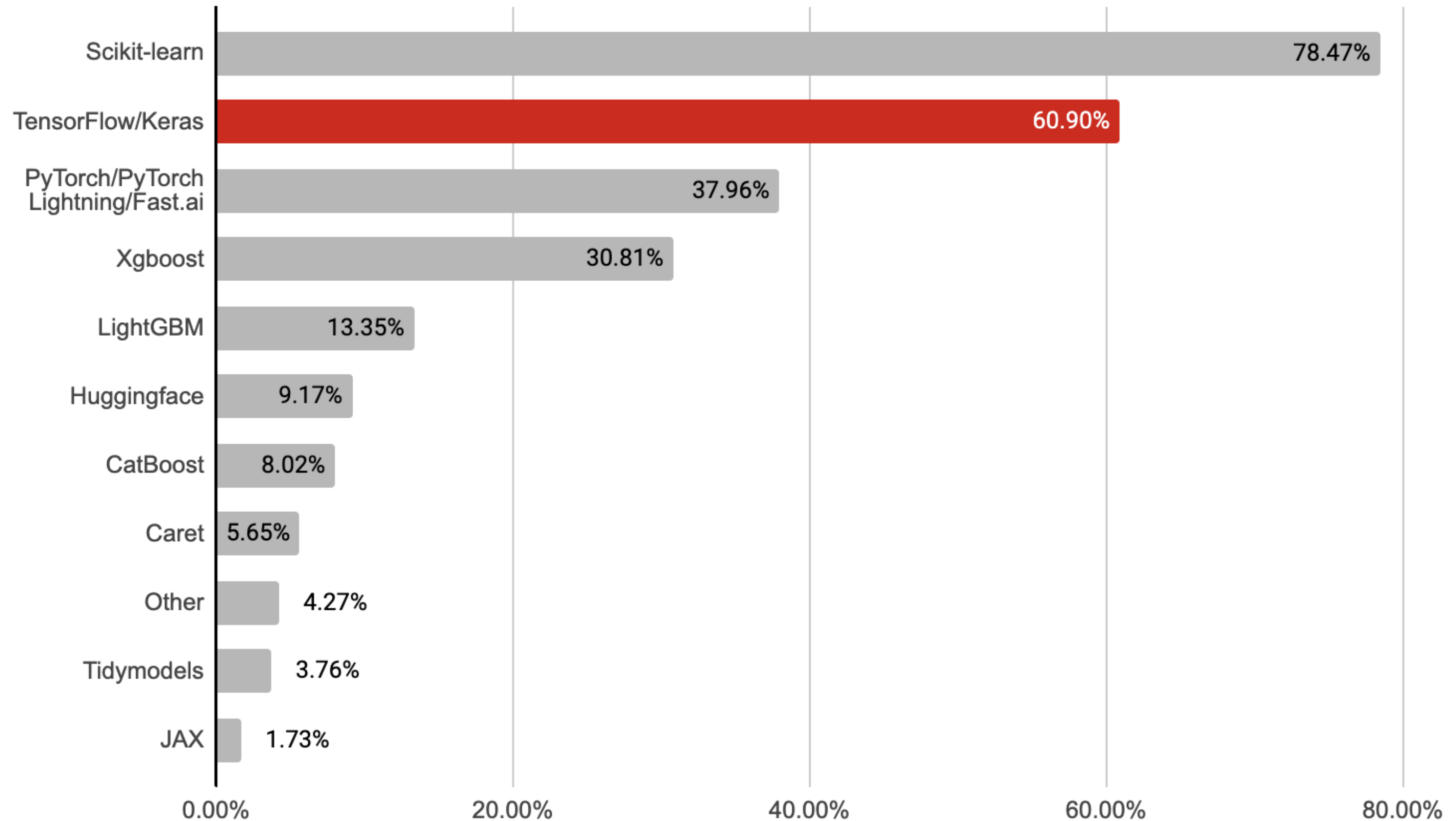
#### Puntos Clave:

- Interfaz de alto nivel para redes neuronales.
- Puede correr sobre TensorFlow, Microsoft Cognitive Toolkit, Theano, etc.
- Fácil de usar y aprender.
- Soporta redes neuronales convolucionales y recurrentes.
- Apto para investigación y desarrollo.
- Extensa documentación y comunidad.
- Rápido prototipado.
- Soporte para múltiples GPUs.

#### Ventajas:

- Desarrollo rápido y prototipado.
- Fácil de aprender y usar.
- Flexible y Modular.

## 2022 Machine Learning & Data Science Survey by Kaggle: library usage (N=14,531)





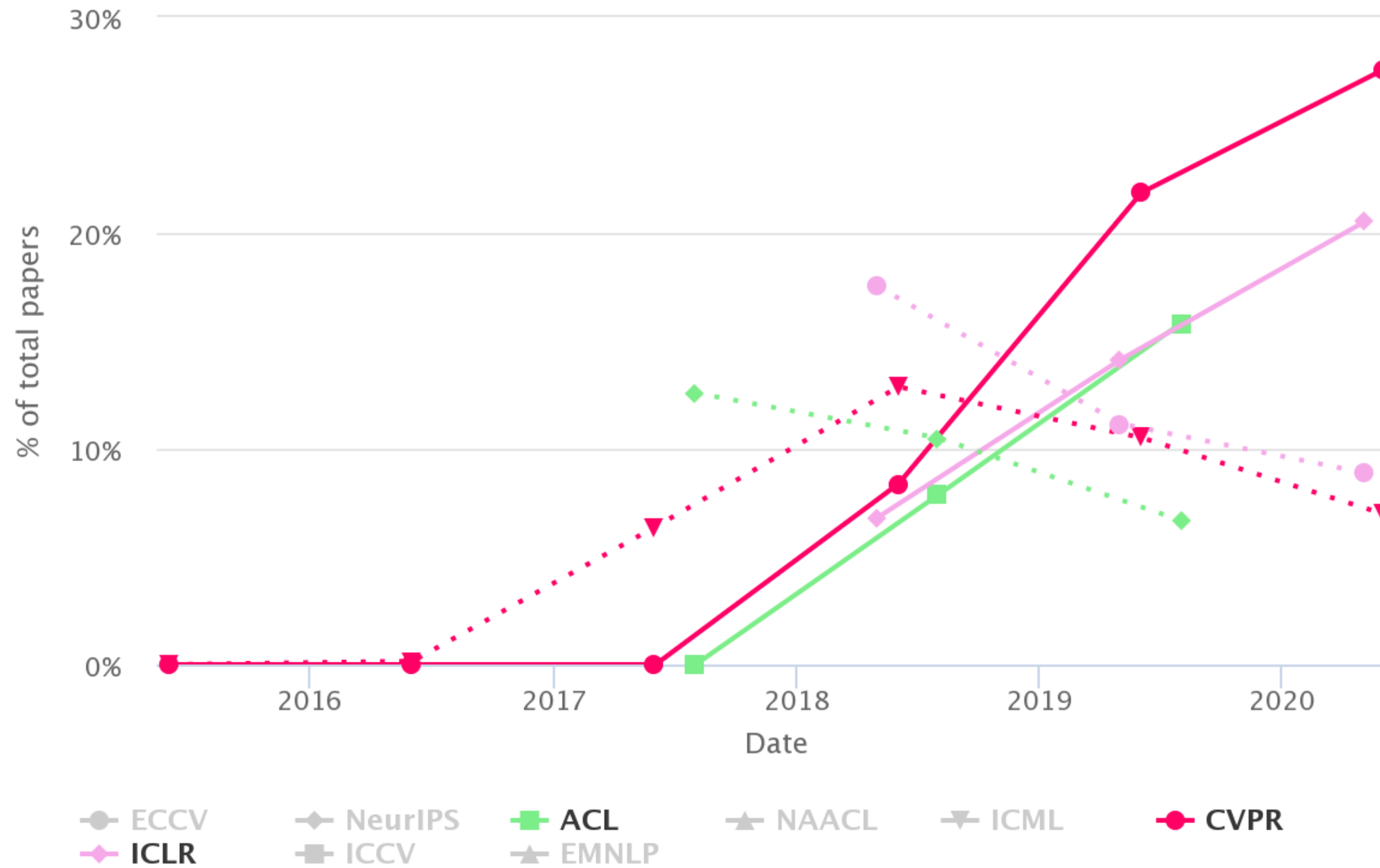
#### Puntos Clave:

- Soporta programación en múltiples lenguajes.
- Escalable y eficiente.
- Soporta entrenamiento distribuido.
- Flexible y permite definición dinámica y simbólica de modelos.
- Optimizado para rendimiento y eficiencia.
- Ofrece Gluon para desarrollo fácil y rápido.
- Amplia documentación y tutoriales.
- Soporta computación en el borde y en el dispositivo.

#### Ventajas:

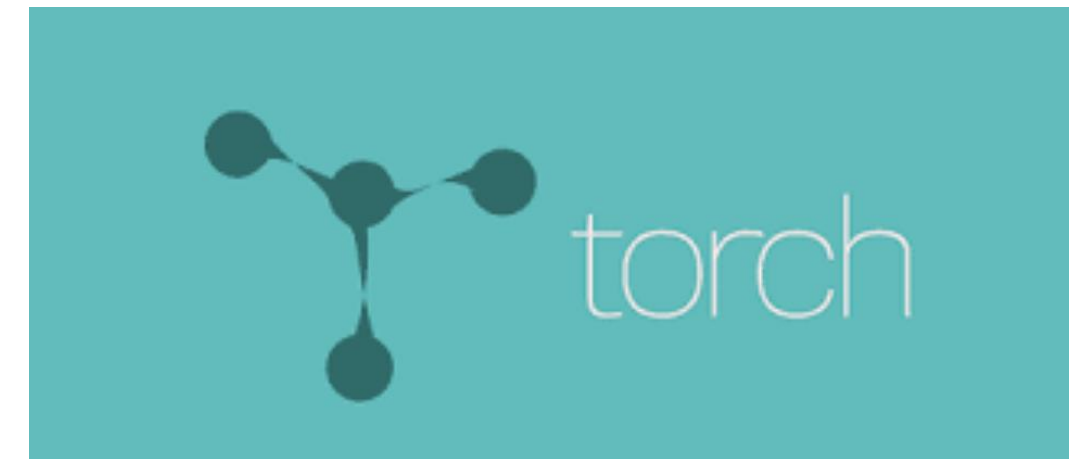
- Multi-lenguaje y Multi-plataforma.
- Escalable y Eficiente.
- Soporte para Entrenamiento Distribuido.

## PyTorch (Solid) vs TensorFlow (Dotted) % of Total Papers





theano



03

DEMO TIME!!

「muchas gracias.」